

SystemSim

Interview Preparation Guide

Architecture, decisions, selling points, and deep-dive reading — compiled from the project's own CLAUDE.md files, BACKEND_LOG.md, docs/AI_INTEGRATION.md, docs/RAG.md, docs/spec.md, PRODUCT.md and DESIGN.md.

Part 1 — What SystemSim Is & Why It's a Strong Portfolio Piece

The elevator pitch

SystemSim is an interactive distributed-systems learning platform. Users design systems visually on a canvas (React Flow), simulate load against their design, see bottlenecks highlighted with real numbers, study real company incident case studies, and take scored challenges against reference architectures. It targets software engineers actively prepping for system design interviews — time-pressured, technically confident, outcome-focused.

Why it's a good interview artifact: it isn't a CRUD app. It has a hand-designed simulation engine with real modeling tradeoffs, a multi-provider AI subsystem with a RAG pipeline, a full auth system built from scratch, and a documented decision history (CLAUDE.md, BACKEND_LOG.md) that shows *engineering judgment over time*, not just a finished feature. Interviewers reward candidates who can narrate the "why" behind a decision and the failure modes they hit — this project is built to make that easy because the reasoning is written down as you went.

The three product modes

- **Sandbox** — free canvas: drag/connect/configure 14 components, set load, simulate, see bottleneck highlights + tradeoff scores. This is the hero surface — "the canvas is the hero," UI chrome recedes.
- **Challenges** — structured scenarios (e.g. "Design a URL Shortener"); user builds a design and gets scored (0–100) against a reference architecture, with concrete numeric feedback.
- **Case Studies** — real company incidents (Discord, Twitter, Slack, Shopify, Instagram, Netflix, Prime Video), structured as Problem → Solution → "Simulate This" (opens a pre-loaded canvas reproducing the *before* state).
- **Learn Roadmap** (added later, became the front door) — a 76-lesson System Design Roadmap, AI-restructured original content, read like a docs site with Mermaid diagrams.
- **Floating AI Assistant** — a chat bubble on every page, context-aware (case study / sandbox graph+result / general), RAG-grounded over the platform's own content, with citation chips.

Selling points — the "so what" a recruiter or interviewer cares about

- 1 **A real simulation engine, not a toy.** Strategy-pattern components, load-splitting semantics, failure injection, cycle detection, accounting invariants (`passed + absorbed + dropped == incoming`) — this is systems-modeling code, defensible line by line.
 - 2 **Multi-provider AI architecture with clean isolation.** Three LLM providers (Claude, Gemini, Groq) have each been swapped in and out of production features *without touching call sites* — proof the abstraction seam is in the right place, not over-engineered.
 - 3 **A production-shaped RAG pipeline**, not a demo — chunking strategy, embedding asymmetry, citation-truthfulness bug found and fixed live, retrieval-failure degradation contract, index idempotency.
 - 4 **Auth built from scratch** (bcrypt + JWT, ownership enforced in the app layer) after a deliberate swap away from Supabase — a concrete "why we didn't just use a managed service" story.
 - 5 **Operational war stories, documented as they happened.** A 76-lesson AI ingest batch that survived a daily quota wall and a mid-run process death — a genuinely good "tell me about a time a system failed and how you handled it" answer, because it's literally logged with timestamps.
 - 6 **A real design system with recorded decisions**, including reversals (light theme added, then explicitly removed — with the reasoning for both).
 - 7 **Everything is decision-logged.** `CLAUDE.md`, `backend/BACKEND_LOG.md`, `docs/AI_INTEGRATION.md`, `docs/RAG.md`, `frontend/DESIGN.md` — five separate living documents that make "why did you do X" answerable from memory instead of guesswork.
-

Part 2 — Stack & Top-Level Architecture

Stack summary

- **Frontend:** React + Vite, React Flow (canvas), Tailwind CSS, shadcn/ui, Zustand (state).
- **Backend:** FastAPI (Python), NetworkX (graph traversal), BeautifulSoup (scraping, stubbed).
- **DB/Auth:** self-hosted Postgres + custom FastAPI auth (SQLAlchemy + Alembic + JWT). Local via docker-compose; AWS planned. (*Swapped away from Supabase — see Decisions below.*)
- **AI:** split by feature across three providers — explainer + roadmap ingest on **Groq**; case-study mentor + floating chat on a **Claude** → **Groq fallback chain**; RAG embeddings on **Gemini** (Groq has no embeddings endpoint).
- **Content rendering:** react-markdown + remark-gfm + rehype-highlight for long-form roadmap lessons, mermaid for flowcharts.
- **Hosting (planned):** Vercel (frontend), AWS (backend + Postgres) — was Render/Supabase.

Architectural decisions worth being able to defend

- **Simulation is stateless.** No server session — the full graph is sent with every `/simulate` request. WHY: trivially scalable, simple to reason about, easy to test in isolation. No session state means no cache-invalidation problem for simulation runs at all.
- **Simulation engine uses the Strategy pattern.** Each of the 14 component types is its own class implementing one interface: `process(load_rps) -> (output_rps, status)`. Adding a 15th component is one new class and zero traversal edits — the classic "open for extension, closed for modification" argument, and it's true here, not aspirational.
- **Challenges & case studies are JSON data, not code.** Content authored without redeloys; keeps the engine logic generic. Deliberate exception: the 76-lesson Learn Roadmap lives in the DB (`roadmap_lessons` table) instead, because it's long-form, growing content that wants per-lesson queries and published/draft state — a blob-of-JSON model doesn't fit that shape. This is a good "when do you choose config-as-code vs. config-in-DB" talking point: the answer tracked to query/growth patterns, not dogma.
- **Case studies and roadmap lessons are AI-structured, never raw scraped/verbatim content.** Copyright-safety principle applied consistently: source material (news articles for case studies, a third-party

paid course series for the roadmap) is fetched only as reference input at ingest time, cached to a gitignored directory, never committed, and the model rewrites it into original content. Nothing verbatim is stored or served.

- **All frontend network calls go through** `frontend/src/api/`. No fetch anywhere else in the app. WHY: one place to add auth headers, base URLs, error handling — and, as a side benefit, an easy structural guarantee for the "frontend never talks to an AI provider directly" security property (see Part 5).
- **Zustand split into slices:** `canvas`, `simulation`, `ui`, `auth`. Avoids one giant store; each screen only touches the slice it needs.
- **Backend routes are thin.** Business logic lives in `engine/` and `services/`; routes parse input, call a service, shape the response. Kept to roughly ≤ 30 lines each.
- **Canvas is optimistic.** UI updates instantly on interaction; the simulation result arrives async and reconciles node styling — a deliberate UX call so the tool never feels laggy even though the backend is a real network round trip.

The auth swap — Supabase to self-hosted

DECISION: rolled a custom auth system on self-hosted Postgres instead of using Supabase Auth + Row-Level Security. Two stated reasons: **learning value / interview talking points** (being able to explain how real auth is built, not "we used a vendor"), and **full control** for self-hosting on AWS later.

What was actually built:

- **Password hashing:** bcrypt via `passlib` (`services/auth.py`).
- **Tokens:** HS256 JWT, access-token only, 60-minute expiry, sub claim = user id. No refresh token yet (documented as planned). Sent as an `Authorization: Bearer` header, not an httpOnly cookie (also documented as planned, pending frontend wiring).
- **Schema:** SQLAlchemy models (`users`, `designs`, `challenge_attempts`, plus later `roadmap_lessons`, `rag_chunks`, `ai_messages`), migrated with Alembic, UUID string primary keys.

The consequence that matters most: with Supabase's Row-Level Security gone, "a user can only touch their own rows" is no longer a database-enforced guarantee — it now lives entirely in the FastAPI application layer. `get_current_user` (in `dependencies.py`) resolves the caller from the JWT, and **every** `designs` / `challenge_attempts` query must explicitly filter by `user_id`. This is called out explicitly as a risk: "forgetting the filter = a data leak that RLS used to catch for us." Cross-user access returns 404 (not 403) so a design ID's mere existence doesn't leak. This is a genuinely good interview point about the tradeoff of leaving a managed-security feature behind — you gain control, you take on an ongoing discipline burden.

Part 3 — Simulation Engine Deep Dive (the "hand-built" core)

This is the piece explicitly protected from being auto-generated for a long stretch of the project ("Satyam writes this by hand") — treat it as the centerpiece of any system-design interview conversation about this project.

The 14 components

Client, API Gateway, Load Balancer, App Server, Worker/Consumer, SQL Database, NoSQL Database, Object Storage, Cache (Redis), Message Queue, CDN, Search Index, Rate Limiter, DNS — grouped into categories
Ingress · Compute · Storage · Async.

Three files, three responsibilities

File	Owns
engine/components.py	The 14 component strategies + their config defaults
engine/simulation.py	Graph building, traversal, failure injection, output assembly
engine/tradeoffs.py	The 0–100 tradeoff heuristics

The core modeling decision: load is a pair, not a number

`WorkLoad(read_rps, write_rps)` — the split happens once, at the client nodes, from the request's `workload.read_pct` (default 80/20 read/write). This single decision is what makes the interesting per-component semantics possible:

- **Cache/CDN** absorb `hit_rate%` of **reads only**; writes always pass through. Absorbed traffic counts as *served*.
- **SQL Database** sends all writes to the primary (`primary_capacity`) and fans reads across replicas (`replicas × read_capacity_per_replica`). Utilization reported is the **worse of the two paths** — either saturating is an incident. With zero replicas, the primary still serves reads at one replica's rate (a documented simplification: "a lone primary still answers SELECTs").
- **Search Index** has separate, much smaller write capacity than read capacity — modeling the classic "indexing lags under write pressure" behavior.

`workLoad` is a **frozen dataclass** deliberately: load values get combined and scaled repeatedly as they flow through the graph, and immutability eliminates a whole class of aliasing bugs where two nodes accidentally share and mutate the same object.

The accounting invariant

Every component strategy buckets its incoming load into exactly three outcomes: **passed** (forwarded downstream), **absorbed** (served right here — cache hits, queue buffering), **dropped** (rejected or lost). The invariant `passed + absorbed + dropped == incoming` always holds. Achieved throughput is computed as requested minus the sum of dropped (clamped) — so a **lagging queue doesn't hurt the headline throughput number** (nothing was lost, it's just late), but a rate limiter *does* hurt it (it actually rejected requests). This distinction — "late" vs. "lost" — is a genuinely subtle systems point worth explaining out loud in an interview.

Traversal

`NetworkX DiGraph`, processed in **topological order** (a multi-source BFS over a DAG collapses to exactly this), so every node sees its complete incoming load before it runs — no partial updates, no re-visits. Two routing rules live in the traversal layer specifically because they're about *edges*, not node behavior:

- 1 **Load-splitting rule.** A `load_balancer` divides its output **evenly** across its successors; every *other* node type sends its **full** output to **each** successor. This models the real-world default that "every request touches both downstream systems it's wired to" — fan-out from an app server to a cache and a queue means both see the whole stream. If you actually want traffic divided between two paths, you must explicitly insert a load balancer. This is called out as *the teaching point* of the simulator.
- 2 **Queue drain discovery.** A `message_queue` needs to know how fast its downstream workers can drain it ($\sum \text{instances} \times \text{consume_rps}$, respecting each worker's own failure-mode-adjusted capacity). If produce-rate exceeds drain-rate, the queue **buffers** the excess (absorbed, not dropped), emits a lag warning, and its status goes to `warning`.

Edge cases handled explicitly: cycles are detected and broken edge-by-edge with a warning (never an infinite loop); disconnected nodes report healthy-but-idle with a warning; multiple client nodes split the requested load evenly; dangling edges from a mid-edit canvas state are skipped with a warning instead of crashing the simulation.

Failure injection

Failure modes mutate a node's **effective config** before its strategy runs, so the strategies themselves stay failure-agnostic (no `if failure_mode` branches scattered through fourteen classes):

- `node_crash` → capacity to 0, status `failed`, all load lost (short-circuits).
- `slow_node` → latency $\times 10$.
- `cache_miss_spike` → `hit_rate` forced to 10%.
- `queue_backup` → drain rate $\times 0.5$.
- `replication_lag` → stale-read warning (modeled as a freshness problem, not a throughput problem — a deliberate distinction).
- `traffic_spike` → global $\times 2$ on requested load.

Status thresholds and output

Utilization <80% **healthy**, 80–100% **warning**, >100% **overloaded**; failed only via a crash injection. Bottlenecks are the overloaded/failed nodes, hottest first. Critical-path latency is computed via **dynamic programming over topological order** as the **longest** root-to-sink path — deliberately the *worst-case* path (e.g. the cache-miss path), "because that's the latency you actually design around." Output matches the locked spec's schema exactly, plus one addition the frontend needed: `node_metrics` (in/out RPS, capacity, utilization %, latency per node) for the properties panel and node badges.

Tradeoff heuristics (`engine/tradeoffs.py`)

0–100 scores derived from graph shape plus simulation outcome:

- **Availability** — rewards a load balancer, replica counts, multi-instance components; penalizes single points of failure.
- **Consistency** — charged for caches present, replication lag, NoSQL eventual-consistency choices.
- **Scalability** — headroom at current load plus presence of horizontal-scaling primitives.
- **Latency** — scored against the computed critical path.
- **Cost / Complexity** — low/medium/high chips from component count, instance totals, distinct component types.

Deep-dive reading if you want to go further

- *Designing Data-Intensive Applications* (Kleppmann) — chapters on replication and partitioning map almost directly onto the SQL DB / NoSQL DB component semantics here (primary/replica split, sharding).
 - Queueing theory basics (Little's Law, M/M/1) — useful to be able to explain *why* "produce > drain ⇒ buffering" is the correct qualitative model even though the engine doesn't do a full queueing-theory simulation.
 - The CAP theorem and PACELC — directly relevant to the consistency/availability tradeoff scores and to case-study material (the Learn Roadmap and mentor both reference it).
 - NetworkX topological sort / DAG algorithms documentation — to be fluent narrating *why* topo-order traversal is the right choice over naive BFS with revisits.
-

Part 4 — Backend, Database & API Architecture

Layout

- `main.py` — app entry, CORS, router registration only. No business logic.
- `routes/` — thin: parse request, call a service/engine, shape response.
- `engine/` — the simulation core (Part 3).
- `services/` — Claude/Gemini/Groq API clients, `auth.py` (bcrypt + JWT), scoring, ingestion, RAG (`rag.py`, `embeddings.py`, `rag_index.py`, `mentor.py`, `chat.py`).
- `db/` — SQLAlchemy `base.py` + `session.py`; `alembic/` for migrations.
- `models/` — `user`, `design`, `challenge_attempt`, `roadmap_lesson`, `rag_chunk`, `ai_message`.
- `dependencies.py` — `get_current_user` (Bearer JWT to User); ownership-scoped routes must filter by the resolved `user_id`.
- `data/` — `challenges.json`, `case_studies.json`, `roadmap_curriculum.json`, `seed_urls.json`.

Full route map

Route	Auth	Notes
POST <code>/simulate</code>	—	Pydantic validates (≥ 1 client, ≤ 60 nodes); engine <code>ValueError</code> maps to 422
GET <code>/challenges</code> , GET <code>/challenges/{slug}</code>	—	<code>reference_graph / bonus_components</code> never leave the server
GET <code>/challenges/{slug}/solution</code>	—	Ungated server-side (it's teaching material); the <i>frontend</i> only surfaces it after a scored attempt
POST <code>/challenges/{slug}/attempt</code>	optional	anonymous = score only; signed-in = persisted

Route	Auth	Notes
GET /challenges/{slug}/attempts, GET /me/attempts	required	own history, dashboard
GET /casestudies, GET /casestudies/{slug}	—	published only
/designs CRUD	required	ownership isolation, see Part 2
GET /auth/me + register/login	—	bcrypt + HS256 JWT
GET /roadmap, GET /roadmap/{slug}	—	module-grouped list + full lesson with prev/next
POST /ai/explain	—	Groq; 503 if unavailable
POST /ai/mentor	—	Claude→Groq chain, RAG-grounded
POST /ai/chat, GET /ai/chat/history	required	RAG-grounded, DB rate-limited
POST /admin/ingest	—	case-study scraper pipeline (stubbed; content is curated JSON today)

Database schema

- `users` (id, email, name, avatar_url, created_at)
- `designs` (id, user_id→users, title, mode, graph_json jsonb, timestamps)
- `case_studies` (id, slug, company, title, source_url, problem, solution, components, scale_context jsonb, lessons, starter_graph jsonb, difficulty, tags, published)
- `challenges` (id, slug, title, description, requirements, reference_graph jsonb, difficulty, tags)
- `challenge_attempts` (id, user_id→users, challenge_id→challenges, graph_json jsonb, score, feedback jsonb, attempted_at)
- `roadmap_lessons` (day unique, slug, module, title, summary, body_md, key_takeaways jsonb, interview_angle, tags, diagram_refs, published, timestamps) — deliberate DB exception, see Part 2.
- `rag_chunks` — chunked+embedded platform content (Part 5).
- `ai_messages` — floating chat history, scoped by (context_type, context_slug).

Dev/prod split: SQLite is the zero-infrastructure dev fallback (`DATABASE_URL` defaults to `sqlite:///./systemsim.db`); Postgres is the production target. `Base.metadata.create_all` covers the SQLite path; Postgres deployments run Alembic migrations (a no-op on tables that already exist). A concrete, tell-able story: Alembic's `versions/` directory was retrofitted after the fact — nobody had generated a migration while the schema evolved organically under `create_all`. The fix pattern (documented, and worth repeating verbatim in an interview): **autogenerate a migration against an empty throwaway database** to capture "everything is new" as the initial migration, verify it applies cleanly to a second empty DB, then `alembic stamp head` (not `upgrade`) against the real, already-correct dev database — `stamp` records "you're at head" without re-running `CREATE TABLE` against tables that already exist. `upgrade` on an already-correct DB would have failed.

Scoring rubric (`services/scoring.py`)

100 points, weighted so both "used the right pieces" and "survived the load" matter:

- **35 pts** required components (proportional; missing types produce specific feedback lines)
- **15 pts** connectivity sanity (client present, traffic reaches storage, nothing floating disconnected)
- **40 pts** *simulated* performance — the user's actual graph is run at the challenge's load: 20 throughput ratio, 10 no overloaded/failed nodes (–5 each, with metric-based feedback), 10 latency budget
- **10 pts** bonus concepts (e.g., a cache present on a read-heavy problem)

A regression worth knowing: the rubric was tightened after a bug where a **disconnected** set of "correct" components could score 80 despite nothing actually flowing — performance points now require traffic to genuinely reach the graph, not just be present on the canvas.

Content strategy and its CI safety net

Content (challenges, case studies) is authored as JSON, loaded with `tru_cache`. A CI-enforced "editorial safety net": tests assert every reference graph simulates clean **and** scores ≥ 80 on its own challenge, and every case-study starter graph at least runs without error. **Bad content fails the build, not the live demo** — a good example of testing content as rigorously as code in a project where content quality is part of the product.

Testing posture

70 backend tests, all green, run against a throwaway SQLite file (no Docker, no API keys needed): engine tests (one per documented rule — cache math, load-balancer split vs. fan-out, SQL write bottleneck, rate-limit capping, queue lag semantics, every failure-injection mode, disconnected nodes, cycle breaking), content/scoring tests (the editorial safety net), API tests (auth roundtrip incl. 409/401, designs CRUD with cross-user 404 isolation, simulate happy/422 paths, AI 503 fallback paths, RAG chunker/retrieval/citation behavior, `/ai/chat` rate limiting).

Part 5 — AI Architecture: Provider Isolation, Structured Output, and RAG

This is the single richest part of the project for a "walk me through a system you designed" conversation — it has a real multi-provider migration history, real production failure handling, and a real bug found and fixed through live testing (not hypothetical).

Current feature map

Feature	Endpoint	Provider	Service
Simulation explainer	POST /ai/explain	Groq (llama-3.3-70b-ve rsatile)	services/ai_explain.p y → services/groq.py
Case-study mentor (RAG-grounded)	POST /ai/mentor	Claude → Groq fallback chain	services/mentor.py → services/rag.py
Floating global assistant (RAG-grounded, auth required)	POST /ai/chat	Claude → Groq fallback chain	services/chat.py → services/mentor.py
RAG embeddings	offline index + per-query	Gemini (gemini-embeddi ng-001)	services/embeddings.p y
Roadmap lesson ingest	offline batch, complete (76/76)	Groq (qwen/qwen3.6-27b)	services/roadmap_inge st.py → services/groq.py

Provider isolation — the seam that made three vendor swaps painless

Each provider is one self-contained module: `services/claude.py` (Anthropic SDK), `services/gemini.py` and `services/groq.py` (raw REST over `urllib`, deliberately no SDK dependency for those two). **Nothing else in the codebase knows which vendor is behind a feature** — routes import a *feature service* (`ai_explain`, `mentor`), never a vendor client directly.

The migration history, because the "why" is the actual interview answer:

- The original plan pinned all in-app AI to Claude. But the key actually available was a Gemini key.
- 2026-07-16 — roadmap ingest (a one-time content batch) shipped on Gemini as an *isolated* provider precisely so the swap would touch nothing else.

- 2026-07-25 — the simulation explainer moved to Gemini too: it had been 503-ing behind a placeholder Anthropic key since day one, and "a working feature on the key you have" beats "a theoretical feature on the key you don't." Mentor stayed on Claude, waiting on a real key.
- 2026-07-26 — both Gemini-backed features moved to **Groq**, the key with usable quota. The explainer runs `llama-3.3-70b-versatile`, the ingest runs `qwen/qwen3.6-27b`. The swap really was what the seam promised: one import line changed in each feature service, plus the route's exception mapping; **the tests passed unchanged**.

The point to be able to say out loud: provider choice is a *deployment detail*, not an architecture decision, because the seam sits at the feature-service level, not the vendor level. Each provider raises its own `AIUnavailable` exception; the route maps every provider's failure onto an identical HTTP 503 contract, so the frontend never knows or cares which vendor is behind a given call.

Model IDs live in exactly one place per provider (`GROQ_MODEL/GROQ_INGEST_MODEL`, `GEMINI_MODEL`, `CLAUDE_MODEL`, all env-driven, read once in that provider's module) — because model churn is constant (the original spec's pinned Sonnet model id was outdated within months of writing), and a model upgrade should be a config change, never a code hunt.

Context-scoping — the "no open-ended chatbot" rule

Project rule: AI calls are **always** context-scoped, carrying concrete state (the user's graph + simulation numbers, or one case study's text) — never a free-text open prompt. Three stated reasons: relevance (the model explains *this* diagram, citing *these* numbers, and can't drift into generic essays), predictable token cost (input size is bounded by the 60-node graph cap), and no jailbreak surface (there's no free-text field for users to wander with; off-topic asks get redirected by the system prompt).

Serialization detail worth knowing: the canvas graph is never sent as raw JSON to the model. It's flattened to a compact line-per-node text form (`id | type | label | config` plus `source -> target` edges), and the simulation result is slimmed to what actually carries signal — statuses, the bottleneck list, per-node metrics **for non-healthy nodes only**, throughput, warnings, tradeoffs. This cuts tokens and, per the docs, produces a format the model grounds against more reliably than nested JSON.

Structured output, not prose parsing

The explainer requests JSON-mode output from Groq (`response_format: {"type": "json_object"}`, guaranteeing syntactically valid JSON) with the response schema injected directly into the system prompt — Groq's *schema-enforced* structured outputs only cover its GPT-OSS model family, so llama/qwen get the injected-schema-plus-JSON-mode approach instead, backstopped by application-level `_validate()` that shape-checks the parsed output, drops malformed entries, and caps result sizes.

```
{ "summary": "...",
  "bottlenecks": [{ "node_id": "...", "label": "...", "why": "...", "fix": "..."}],
  "suggested_fixes": ["..."] }
```

A correction worth repeating verbatim, because it's a mature take on provider guarantees: JSON mode makes invalid JSON *rare*, not impossible — one roadmap ingest day (day 67) hit Groq's own server-side HTTP 400 `json_validate_failed`. The real safety net was never the mode's guarantee; it was that the ingest pipeline already treats every unit of work as independently retryable and skips-not-aborts on failure. **Don't oversell a provider guarantee in an interview — describe the retry/skip design that makes its occasional failure harmless instead.**

Security: the key never leaves the backend

GROQ_API_KEY / GEMINI_API_KEY / ANTHROPIC_API_KEY live only in backend/.env (gitignored). The frontend never calls an AI provider directly — all AI traffic goes browser → FastAPI → provider, and the browser bundle contains no key, no provider URL, no model name. This is enforced structurally, not just by convention: the project rule that every network call lives in frontend/src/api/client.js makes it a one-file audit. Beyond secrecy, this is also the chokepoint where prompt construction happens (users can't tamper with the system prompt) and where future rate limiting, auth, caching, and logging will live.

Vendor-specific gotchas actually hit in production (great "debug a live issue" answers)

- **Gemini:** internal "thinking" tokens count against `maxOutputTokens` — a 1024-token cap silently truncated responses *mid-JSON*; raised to 4096.
- **Groq:** Cloudflare fronts `api.groq.com` and returns 403 (error 1010) against urllib's default `Python-urllib/x.y` User-Agent — fixed by sending an explicit UA. Also, the qwen reasoning model burns its completion-token budget on internal "thinking" before ever emitting JSON, which fails Groq's JSON-mode validation outright — fixed by sending `reasoning_effort: "none"` on ingest calls and stripping any leaked `<think>` prefix defensively.
- **Groq rate limits, three dimensions:** requests-per-minute, tokens-per-minute (both retryable — Groq returns a `retry-after` header, honored, capped at 120s, exponential backoff otherwise), and tokens-per-day (**not** retried in-process — see the batch-resilience story below). Groq also rejects oversized requests outright with HTTP 413 (`prompt + max_completion_tokens` checked against the per-minute cap at request time) — **never retryable**, so the ingest pipeline budgets both sides of every request in advance (truncates source material at a newline boundary when oversized, sizes the output cap to the remaining token budget).

RAG architecture (`docs/RAG.md`) — the deepest system in the project

Why RAG at all: the pre-RAG mentor stuffed one case study's full text into every prompt and hoped the model free-recalled the rest of the platform's content. Two problems: a **grounding gap** (a Day-N lesson teaching exactly the asked-about concept sat unused in the database while the model answered from general training), and a **token ceiling** ("put everything in the prompt" doesn't scale past one document — retrieval makes prompt cost $O(k)$ regardless of corpus size). RAG also buys **verifiable citations** — every answer ships links to the exact sections it drew on.

```
Ingest (offline, idempotent):
  roadmap_lessons + case_studies.json → chunker → ~500 chunks
  → Gemini embeddings (RETRIEVAL_DOCUMENT) → rag_chunks table

Query (online, per question):
  question + context (case_study/sandbox/none)
  → Gemini embeddings (RETRIEVAL_QUERY)
  → exact cosine over in-process numpy matrix (~500 × 768 float32, cached)
  → top-4, score ≥ 0.45, max 2 chunks per source document
  → prompt assembly (context + passages + question)
  → Claude if configured, else Groq llama-3.3-70b
  → filter sources to only [S#] tags actually referenced in the answer
  → {answer, sources, grounded} (persisted to ai_messages for /ai/chat)
```

No vector database — the math that justifies it

The corpus is ~500 chunks × 768 dimensions × 4 bytes ≈ **1.7 MB**. A single numpy matrix-vector product over that is exact (100% recall) and takes well under a millisecond. An ANN index (pgvector HNSW, FAISS, a

managed vector DB) exists to make search *cheaper than exact* at scales where exact search is slow — and pays for that speed with approximate recall, index build/maintenance overhead, and an infrastructure dependency. Below roughly 50,000 vectors, exact scan stays single-digit milliseconds; adopting a vector DB at 500 chunks would be paying ANN's costs while capturing none of its benefit. Documented trigger to revisit: corpus > ~50k chunks or p95 retrieval > ~20ms. Documented upgrade path: swap the JSON embedding column for a pgvector `vector(768)` column with an HNSW index, replace the numpy dot product with `ORDER BY embedding <=> :q LIMIT k` — chunking, prompting, citations, and the API contract stay untouched because `rag.retrieve()` is the one seam that changes.

Embeddings on Gemini, generation on a Claude→Groq chain — constraint-driven, stated honestly

Groq (the quota-bearing generation key) has **no embeddings endpoint**; Gemini's embeddings tier is free and the key is real; Anthropic was still a placeholder key at RAG build time. So embeddings are isolated in `services/embeddings.py` (same provider-isolation discipline), and generation is a **chain, not a load balancer** — Claude when a key exists (the original quality preference), else Groq. A subtlety worth naming explicitly: **the embedding model choice is sticky in a way generation swaps are not** — stored vectors are only comparable to queries embedded by the *same* model, so swapping generation providers is a config change, but swapping the embedding provider means re-embedding the entire corpus. Cheap at 500 chunks (one forced rebuild), but that asymmetry is exactly why the two live in separate modules with separate env vars rather than one "AI provider" abstraction. Also worth knowing: Gemini's retrieval embeddings are **asymmetric** — short questions are embedded with a different task type (`RETRIEVAL_QUERY`) than long passages (`RETRIEVAL_DOCUMENT`) so they land near each other in vector space; using one task type for both silently degrades recall, and the provider module encodes the right one per call site so callers structurally can't get it wrong.

Chunking on document structure, not fixed windows

`chunk_markdown()` cuts at `##` heading boundaries (the semantic boundaries the content already has) so a chunk never mixes two topics. Oversized sections split on paragraph boundaries with one paragraph of overlap (so a sentence straddling a cut point exists in both chunks); fenced code blocks are stashed and restored around any split so a chunk never contains half a code block. Target size ≈1,600 characters (~400 tokens) — small enough that a hit is precisely *about* the question, large enough to carry a complete thought.

Two cheap-but-high-value refinements: **context-prefixed embedding text** (a chunk saying "it uses consistent hashing" embeds poorly in isolation, so the text sent to the embedder is prefixed with `"Day 12: Caching — Cache invalidation\n\n<passage>"` while the *clean* passage — without the prefix — is what's stored and shown in the prompt/citation — the classic fix for chunking's "pronouns lose their antecedent" failure mode); and **excluding reader-instruction boilerplate at chunk time** — every roadmap lesson ends with a near-identical "Try it in the sandbox" activity nudge that chunks and embeds fine but is an instruction to the reader, not explanatory content. It was found live, surfacing as a citation chip next to real technical sources, diluting them — fixed by skipping any section whose heading starts with that phrase, dropping 48 of 547 chunks with **zero re-embedding cost** (the diff-based index build treats removed sections as pure deletions).

Retrieval policy

Top-4 chunks, cosine floor **0.45**, max **2 chunks per source document** (diversity — one lesson can't monopolize the context window), and the current case study's own chunks are explicitly excluded from its own mentor query (its full text is already in the prompt; re-retrieving it would waste slots on duplicates). The floor matters more than top-k, because retrieval *always* returns nearest neighbors whether or not they're actually relevant — without a floor, an off-topic question decorates the prompt with noise and can invite the model to cite irrelevant content.

The citation-truthfulness bug — a genuinely great "tell me about a bug you found" story

Found via **deliberate live testing before building further on top of a shipped feature**. Asked the mentor, on the Discord case study, "What's your favorite pizza topping?" It correctly declined and redirected — but the response still came back grounded: `true` with **four citation chips for unrelated roadmap lessons**, and none of their `[S#]` tags appeared anywhere in the actual answer text. Root cause: retrieval scored those four chunks at 0.52–0.525 — just over the 0.45 floor, which turns out to be roughly the *noise floor* for almost any professional-sounding English text against a technical corpus. The model correctly ignored all four in its answer; the API still returned them because `sources` reflected **what retrieval found**, not **what the answer actually used**.

The fix, and why it's the right fix and not a threshold hunt: raising `MIN_SCORE` would only relocate where false positives happen, because "is this cosine score high enough" and "did the model actually rely on this passage" are fundamentally different questions — and the second one is exactly what a citation is *for*. `_cited_sources()` now regexes the model's own generated text for `[S(\d+)]` tags and keeps only the chunks whose tag was actually referenced; `grounded` is derived from that survivor list, never from `bool(chunks)`. This is more robust than any score cutoff, because it never has to be right about relevance in advance — it just checks what actually happened. Regression-tested (`test_mentor_drops_retrieved_but_uncited_sources`).

Failure contract — RAG degrades, it never becomes a new outage

Failure	Behavior
Index empty (fresh deploy, pre-build)	<code>retrieve()</code> returns <code>[]</code> without even calling the embeddings API; answers from context alone, <code>grounded: false</code>
Embeddings API down/quota'd at query time	Same degradation — any retrieval exception is caught in <code>mentor.respond()</code>
Claude unavailable (including a placeholder key returning 401, verified live)	Transparent Groq fallback, <code>provider</code> reported in the response
Both generation providers unavailable	503 — the same contract the frontend already handled pre-RAG

Stated invariant: **adding RAG must not add a new way for the mentor to be down**. Retrieval failing loses grounding quality, never the feature itself.

Index builds are idempotent and content-hash-diffed

`python -m services.rag_index build` diffs the desired chunk set against what's stored, by content hash: unchanged chunks are skipped (no embedding call spent), changed/new chunks are embedded, orphaned chunks are deleted. Editing one lesson re-embeds only that lesson; a crashed build resumes for free; running the command twice in a row is a no-op. This is the same principle that carried the 76-lesson roadmap ingest through its quota wall (below): **the API call is the scarce, rate-limited resource, so the system's job is to never repeat one it doesn't strictly have to**.

The floating global assistant — one core, three context modes, auth-gated for a real reason

`mentor.respond()` takes `case_study` xor `sandbox` xor neither as its context argument — all three modes still run retrieval over the *whole* corpus (sandbox mode means "also ground in what's currently on the canvas," not "skip the platform corpus"). `services/chat.py` layers persistence and a **DB-backed rate limit** on top for the new surface: `SELECT COUNT(*)` over `ai_messages` for `(user_id, role='user', created_at >= now - 10min)`, rejecting past 20 messages with `429 + Retry-After`.

Why auth is required here specifically (stricter than the per-case-study mentor): auth-gating alone is not a rate limit — it's the *precondition* that makes one possible. An anonymous endpoint has no stable identity to count against (IP is spoofable and unfairly punishes users on a shared network); attributing every message to a `user_id` is the only reason a simple count-based cap works at all. No Redis — same reasoning as skipping a vector DB: one indexed COUNT query is faster than the LLM call it's protecting, at this scale. A documented upgrade path exists (a Redis sliding-window counter behind the same function signature) if this ever shows up in a profile.

A resilience detail worth mentioning: the user's message is persisted **before** generation runs, so a provider outage loses the *answer*, never the *question* — an honest "you asked this and didn't get a reply" record beats silently discarding the user's input.

One real cross-database bug, worth remembering generally: SQLite hands back a **naive** (no tzinfo) `datetime` from a `DateTime(timezone=True)` column on read, even though it was written as UTC — only Postgres round-trips timezone info correctly. This crashed the rate-limit's retry-after math (`can't subtract offset-naive and offset-aware datetimes`); fixed with `.replace(tzinfo=timezone.utc)` before doing arithmetic. Worth flagging as a general lesson for any project that runs SQLite in dev and Postgres in prod: **datetime arithmetic on timestamps read back from the DB is not portable across those two engines without an explicit normalization step.**

A bounded general-knowledge fallback — the one deliberate exception to "no open-ended chat"

The project's "AI calls are always context-scoped" rule got one narrow, explicitly-logged carve-out: the mentor's system prompt has a stated PRIORITY ORDER — (1) ground in given context, (2) ground in retrieved platform passages, (3) **only if neither applies**, answer from general engineering knowledge, clearly labeled ("Outside SystemSim's own content, but generally speaking..."). Verified live: asking to "explain CAP theorem in general" now produces a labeled general answer instead of the old bare redirect, while a genuinely off-topic question (the pizza example) still gets redirected, because there's nothing in any of the three priority tiers to answer *from*. The framing worth repeating: this is "answer generally when we have nothing platform-specific," not "always try to answer everything" — retrieval still runs first on every single question.

Deep-dive reading if you want to go further

- Anthropic / OpenAI / Google docs on **structured output / JSON mode / tool use** — to be fluent on the differences between *schema-enforced* generation (Gemini `responseSchema`) vs. *schema-injected-plus-validated* generation (Groq JSON mode) — this project actually uses both, for real reasons.
- **RAG fundamentals:** chunking strategies, embedding task types, and the "why RAG over long-context stuffing" argument — Kleppmann-adjacent but AI-specific; Pinecone's and Anthropic's own RAG engineering write-ups are good primers.
- **Rate limiting patterns** — token bucket vs. sliding window vs. fixed window, and why "requests per minute / tokens per minute / tokens per day" are three genuinely different constraints that need different handling (this project explicitly treats the daily cap differently from the per-minute caps, and can explain why).
- **Idempotency in batch/ETL pipelines** — the roadmap ingest's whole resilience story is really a batch-idempotency story; reading up on idempotent upserts and "checkpoint via queryable state, not process status" (a general distributed-systems pattern, not AI-specific) will let you generalize this story beyond "we called an LLM 76 times."

Part 6 — Operational War Stories (great "tell me about a time" answers)

The roadmap ingest: surviving a daily quota wall and a mid-run process death

The 76-lesson roadmap ingest is the one AI feature that's a **long-running offline batch** (76 sequential model calls) rather than a request/response call — a genuinely different failure surface, and the richest "describe a system failure and how you recovered" story in the project.

The daily-quota wall. Groq's free tier caps the ingest model at 200,000 tokens/day — separate from the per-minute cap the retry logic already handles. The overnight batch hit it mid-run, with Groq reporting a wait of over 30 minutes before the daily counter would allow more calls. The deliberate decision was to **not** stretch the retry loop to cover this: a process blocking for 30–40 minutes on a single HTTP call is its own operational hazard, and the wait time itself shrinks unpredictably run to run. Instead, the failing days were allowed to fail through the pipeline's existing **per-day try/except** (already built so one bad day doesn't abort the whole batch), and simply **re-run once the daily counter rolled over**.

Why this worked with zero bookkeeping: the ingest's `upsert()` keys on a unique `day` column — re-running a batch that includes already-completed days is a no-op for those days. Combined with per-day exception isolation, a batch that dies at day 53 of 76 for *any* reason (daily quota, a dropped connection, the host process itself exiting) needs no record of exactly where it stopped: **query which days already exist, and pass the rest back in**. The interview-ready generalization: for a multi-unit batch job, investing in cheap, verifiable resumability (idempotent writes plus queryable state) beats investing in a retry loop that tries to survive longer — retries handle transient failures, idempotency handles the failure modes retries structurally can't (process death, daily quotas, anything measured in tens of minutes).

Trust the data, not the process status. The batch was also running as a background process when the *development session itself* ended mid-run. On resume, the background task's own completion signal was ambiguous — "may have been running when the process exited," not a real completion record. Rather than guess, the recovery move was to query the actual source of truth directly: `SELECT day FROM roadmap_lessons`, diffed against the curriculum's 76 expected days. This is the same idempotency principle generalized: **when a batch job's own persisted state is more trustworthy than a process supervisor's status string, check the state, not the process**.

Trust but verify, even with JSON mode. Two more days failed on separate later runs, both absorbed by the existing per-day resilience with zero new code: one hit Groq's own server-side HTTP 400

`json_validate_failed` (its JSON-mode generation produced invalid JSON on Groq's end, not ours — the correction to "JSON mode guarantees valid JSON" noted above), the other was an ordinary transient `RemoteDisconnected` network error. Both succeeded on a plain retry.

A validation gap the pipeline didn't catch — and a nuanced fix. Two published lessons (of 76) contained ASCII box-drawing diagrams despite the ingest system prompt's explicit "never ASCII, always Mermaid" instruction. The pipeline validates JSON *shape* but had no check on markdown *style* inside the body text — a model can satisfy the schema and still violate a prose-level house rule. Caught only by a manual post-ingest sweep (searching all 76 lesson bodies for box-drawing glyph characters), not by the pipeline itself. The fix is worth repeating as a nuanced point: one lesson (a folder-tree diagram) converted cleanly to a Mermaid flowchart; the other (a 64-bit Snowflake-ID bit-field layout) was converted to a **markdown table instead**, because no Mermaid diagram type represents a bit map — forcing one would have been worse than the ASCII it replaced. **The interview-ready framing: the fix for "never do X" isn't always "always do Y instead" — sometimes it's "do Y, except when the content genuinely doesn't fit Y's shape," and knowing the difference is the actual skill.**

Final numbers: 76/76 lessons ingested and published; average body 6,412 characters; 65/76 contain a Mermaid diagram (the rest are diagram-free by content, not by gap); zero remaining ASCII art; zero leaked reasoning-model `<think>` tokens.

The Alembic retrofit

Covered in Part 4 — worth restating as its own story: a schema that evolved for weeks under `create_all` with no migration history, retrofitted by autogenerating a migration against an empty database (to capture "everything is new" as the initial revision) and then stamping (not upgrading) the real, already-correct database to align it with that revision without re-running any DDL. A clean, reusable pattern for "we forgot to set up migrations and now we have real data" — a situation almost every backend engineer has actually hit.

`--reload` dev servers racing migrations

Twice during this project, a running `uvicorn --reload` process re-imported `main.py` (and ran `create_all`) at the exact moment a new SQLAlchemy model file landed on disk, creating the table **before** the corresponding Alembic migration ran — so `alembic upgrade` then failed with "table already exists." The recorded rule of thumb: with a reloading dev server running, adding a new model means the table may already exist by the time you go to migrate it, and `alembic stamp` (not a retry) is the reconciliation tool. This happened twice and is logged as a repeatable gotcha, which is itself a good sign of a team (of one) building institutional memory correctly.

Part 7 — Frontend, Design System & Product Decisions

Design identity

Register: Product design — clarity over decoration. Brand personality: **Clean · Educational · Approachable** — "the best professor you ever had," Excalidraw's whiteboard ease crossed with Linear's product discipline. Explicit anti-references: Lucidchart-style bloated enterprise chrome, and the opposite failure mode — LeetCode-style bare utility with no craft or delight.

Five stated design principles

- 1 **The canvas is the hero** — UI chrome recedes; toolbars/panels serve the architecture being drawn, not the other way around.
- 2 **Fast loops build confidence** — simulate instantly, surface bottlenecks directly, minimize clicks between iterations.
- 3 **Opinionated, not overwhelming** — strong defaults so users trust the tool without being buried in configuration.
- 4 **Clarity over cleverness** — every label, score, and result readable at a glance; if a user has to think about what a result means, the UI failed.
- 5 **Earn every interaction** — no decorative chrome; every panel, tooltip, and transition must pay for itself in clarity or delight.

Accessibility baseline: WCAG 2.1 AA, keyboard-navigable canvas controls, bottleneck highlights that never rely on color alone (icons/labels alongside red/green), and `prefers-reduced-motion` respected throughout.

Visual system

Dark-technical is the app's **only** theme (see the light-theme decision reversal below). Six semantic color tokens (base/surface/elevated/dim/ink/muted, plus hairline) are CSS variables, not static hex — mapped through Tailwind's `rgb(var(--color-x) / <alpha-value>)` pattern so opacity modifiers keep working. Brand accent is violet (`#7C5CFF`) fully overriding Tailwind's indigo scale so every existing `indigo-*` class site repaints without a rename. A second accent, mint (`#34E2A1`), is reserved strictly for "healthy / actively simulating" states — never decorative.

Typography: Space Grotesk for display headings only, DM Sans for all body/UI, Space Mono for technical tags/metrics, and Source Serif 4 (font - read) reserved exclusively for long-form reading content (case-study prose, roadmap lessons) so reading-length paragraphs feel like an article rather than a control.

The light theme: added, then explicitly removed — a good "reversing your own decision" story

Full light/dark theming was built in one pass (CSS-variable toggling via a `.light` class, `next-themes`), including a real contrast bug that was **found and fixed as part of building it**: a fixed-dark-background component reused theme-reactive text classes that would have gone dark-on-dark in light mode. Ten days later, the decision was explicitly reversed: light mode and the toggle were removed entirely, because — stated directly — "light mode was extra surface area (a second value for every token, a contrast pass on every new component) for a tool whose primary identity was always dark technical — not worth maintaining as a second first-class theme." The CSS-variable *mechanism* was deliberately kept even after the reversal (tokens just carry one value each now instead of a pair), because it's still the right way to manage six semantic colors regardless of whether a second theme exists. **This is a genuinely mature product story: shipping a feature well, then killing it for the right reason, without ripping out the infrastructure that was still valuable.**

The Learn Roadmap and its front-door promotion

The Learn area (76-lesson roadmap) was originally a secondary feature; a later decision moved it to the front of the site's information architecture (nav order became Learn → Sandbox → Challenges → Case Studies → Dashboard), with interview-prep content demoted from a top-level nav item to a sub-page under Learn. Stated reasoning: "the site is a structured learning platform now, not just a simulator; the nav should lead with the library." This is a real content-strategy pivot, not just a UI tweak, and it's traceable to a specific dated decision in the design log.

Markdown lesson rendering distinguishes **syntax-highlighted code blocks** (VS Code "Dark+" theme via `rehype-highlight`) from **language-less fences**, which are deliberately rendered as terminal/CLI chrome (mac-style dot row, "terminal" label, distinct darker background) — because command output and source code shouldn't look visually identical, a small but real UX judgment call.

Deep-dive reading if you want to go further

- Linear's and Figma's own design-system write-ups (both named as direct visual references) — useful for articulating *why* a small, disciplined token set beats an ad-hoc palette.
 - WCAG 2.1 contrast-ratio guidelines — this project actually measured and fixed real failing contrast ratios (1.4–2.8:1 on white for some status-color text steps), so being able to cite the 4.5:1 body-text / 3:1 large-text thresholds concretely strengthens this story.
 - React Flow's own documentation on custom nodes/edges — relevant background for explaining the canvas implementation choices (one custom node component per system component type, sharing a base).
-

Part 8 — Likely Interview Questions & How to Frame Answers

"Walk me through the architecture."

Start top-down: three product surfaces (Sandbox, Challenges, Case Studies) plus a Learn Roadmap, backed by a stateless FastAPI backend and a React/Zustand frontend, with a hand-built simulation engine as the computational core and a multi-provider AI layer as an enhancement layer that degrades gracefully to "unavailable" rather than breaking the product. Anchor on the fact that simulation is **stateless** (full graph sent per request) — it's the single decision that makes the rest of the backend simple to reason about.

"What's the most interesting engineering decision you made?"

Two strong candidates, pick based on what the interviewer seems to want:

- **Systems-modeling angle:** the `workLoad(read_rps, write_rps)` split and the passed/absorbed/dropped accounting invariant — a small design choice that made an entire category of realistic component behavior (cache hit-rates, replica read/write asymmetry, "buffered vs. dropped") fall out naturally instead of being special-cased per component.
- **AI/systems angle:** the provider-isolation seam that let three separate live vendor swaps (Claude→Gemini→Groq, twice) happen as one-import changes with zero test breakage — proof the abstraction boundary was drawn in the right place.

"Tell me about a bug you found and how you fixed it."

The RAG citation-truthfulness bug (Part 5) is the strongest answer in the project: it was found by *deliberately trying to break a feature via live testing before building more on top of it* (a good habit to name explicitly), the root cause was subtle (a cosine-similarity noise floor, not an obvious crash), and the fix reflects a mature engineering instinct — not tuning a threshold, but asking a fundamentally better question ("did the model actually use this," not "was this score high enough").

"Tell me about a system failure and how it recovered."

The roadmap ingest's daily-quota-wall story (Part 6). Strong because it has three distinct, chainable lessons: idempotent writes make resumability free, persisted state beats a process supervisor's status string, and retries

and idempotency solve *different* classes of failure (don't conflate them).

"How would you scale this?"

RAG retrieval has a documented, concrete answer already: swap a JSON embedding column for pgvector at the ~50k-chunk trigger, with the entire rest of the pipeline untouched because `rag.retrieve()` is the one seam. For the simulation engine, the honest answer is that it's already $O(\text{nodes}+\text{edges})$ per simulation (a topological traversal) and stateless, so the natural next step is horizontal scaling of the FastAPI process itself, not an engine rewrite. For the database, the auth section's honest gap (no rate limiting yet on `/simulate` or `/ai/*`, tracked as an explicit pre-deploy TODO since early in the project) is worth naming unprompted — it shows awareness of what's *not* done, which reads better than pretending everything is finished.

"What would you do differently / what's still missing?"

Answer from the project's own honestly-tracked open items rather than inventing gaps: no rate limiting yet on `/simulate` or `/ai/*` (flagged repeatedly as the top pre-deploy blocker); no retrieval evaluation set for RAG (chunking/threshold changes are currently judgment, not measurement — named explicitly as "the honest gap"); no refresh tokens for auth yet; Postgres migrations have never been dry-run against real Postgres (only SQLite, due to no Docker in the dev environment); the ingest pipeline has no automated style-conformance check (the ASCII-diagram gap in Part 6) — currently a manual sweep. Being able to list real, specific, currently-true gaps is far more credible than claiming a finished product.

General framing advice

Every major decision in this project has a **stated WHY**, not just a WHAT — use that. When asked "why X," answer with the actual documented reasoning (constraint-driven, e.g. "we had a Gemini key, not an Anthropic one," is a *better* answer than a fabricated principled one) and then name the tradeoff it costs. Interviewers consistently rate "I knew the tradeoff and chose it deliberately" above "I picked the theoretically best option," because the former is what actually happens in real engineering work.